

URL Shortener

System Design — Study Notes

Base62 Encoding · Unique ID Generation · Zookeeper
Snowflake IDs · System Architecture · DB Schema

TOPICS COVERED

1. Problem Definition & Requirements

2. Base62 Encoding & URL Length

3. Hash Functions — MD5, SHA1

4. ID Generation — Zookeeper, Snowflake

5. System Architecture

6. Shorten & Redirect Request Flow

7. Database Schema & Design

8. Quick Revision & Interview Qs

Table of Contents

1. Problem Definition & Requirements	3
2. Base62 Encoding & URL Length	3
3. Unique ID Generation Approaches	4
3.1 MD5 / SHA1 Hash Functions	4
3.2 Ticket Server	4
3.3 Snowflake ID Algorithm	4
3.4 Zookeeper Range Allocation	4
4. System Architecture	5
5. Request Flow — Shorten & Redirect	6
6. Database Schema & Design	7
7. Quick Revision & Interview Questions	7

1. Problem Definition & Requirements

Objective: Design a service that takes a long URL and returns a short URL that redirects to the original. Think TinyURL, bit.ly, or LinkedIn's URL shortener.

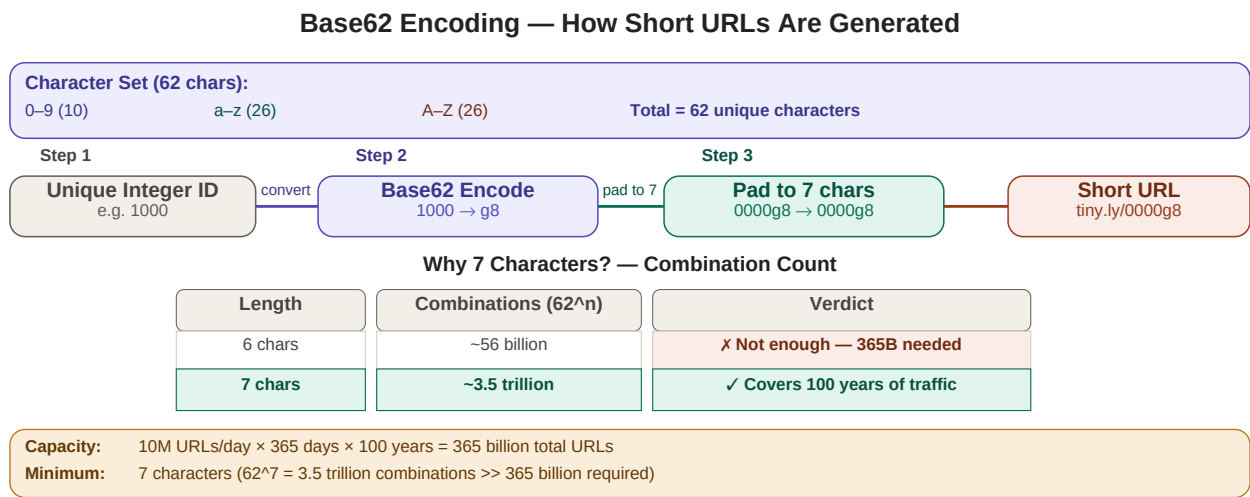
Requirement	Value	Reasoning
Daily URLs	10 million/day	High-traffic production load
Lifespan	100 years	Forward-thinking design
Total URLs	365 billion	$10M \times 365 \times 100$
Short URL length	7 characters minimum	$62^6 = 3.5T > 365B$ required
Character set	Base62 (0-9, a-z, A-Z)	62 unique chars, URL-safe
Read:Write ratio	~100:1 (read-heavy)	Most traffic is redirects
Latency	< 10ms for redirect	User experience critical

Interview tip: Always ask clarifying questions — URL length, traffic volume, expiry support, custom aliases, analytics needed?

The system is read-heavy (redirects >> shortenings) — design the cache and DB indexes accordingly.

2. Base62 Encoding & URL Length

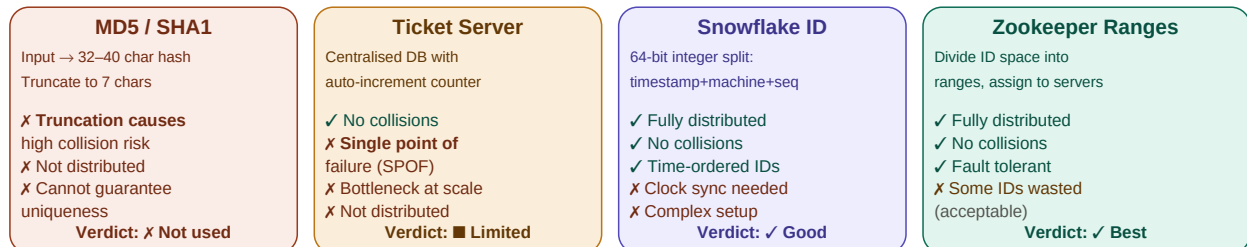
Base62 encoding converts a unique numeric integer into a compact string using 62 characters (0–9, a–z, A–Z). This is the core mechanism for generating short URLs.



3. Unique ID Generation — The Core Challenge

The hardest part of URL shortener design is generating **unique integer IDs at scale** across multiple distributed servers without collisions. There are four approaches:

Unique ID Generation — 4 Approaches Compared



Snowflake ID — 64-bit Layout



Zookeeper Range Allocation



4 approaches: MD5/SHA1 (collisions), Ticket Server (SPOF), Snowflake (distributed time-based), Zookeeper (range allocation)

3.1 MD5 / SHA1 Hash Functions

MD5 produces a 128-bit hash (32 hex chars). SHA1 produces 160-bit (40 hex chars). Both must be **truncated to 7 characters** — this causes hash collisions where different long URLs map to the same short URL. Not suitable.

3.2 Ticket Server (Centralised Auto-increment)

A single database server with an auto-increment counter. Simple and collision-free, but a **single point of failure (SPOF)** and bottleneck at 10M URLs/day. Cannot scale horizontally.

3.3 Snowflake ID Algorithm (Twitter)

Generates a **64-bit unique integer** using: 1 sign bit + 41 bits timestamp (ms) + 10 bits machine ID + 12 bits sequence. Fully distributed, time-ordered, and produces up to 4096 IDs/ms/machine. Minor drawback: requires clock synchronisation.

3.4 Zookeeper Range Allocation (Recommended)

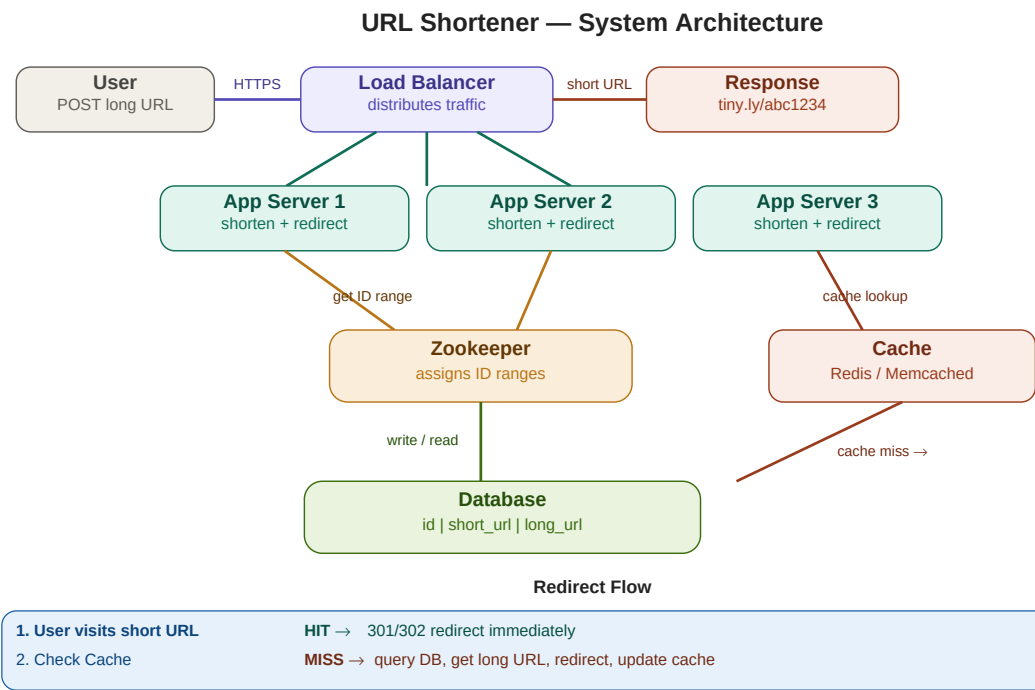
Apache Zookeeper divides the 3.5 trillion ID space into ranges and assigns each range to a server. Each server generates IDs from its assigned range. When exhausted, it requests the next range. **Zero collisions, fully distributed, fault-tolerant.**

Zookeeper example: Server 1 gets IDs 0–1,000,000. Server 2 gets 1,000,001–2,000,000. No coordination needed per request.

Wasted IDs: if Server 1 uses only 500K IDs before restart, 500K are wasted. This is acceptable — 3.5T total IDs is vast.

Best choice for URL shortener: simple, distributed, no clock dependency, easy to reason about.

4. System Architecture

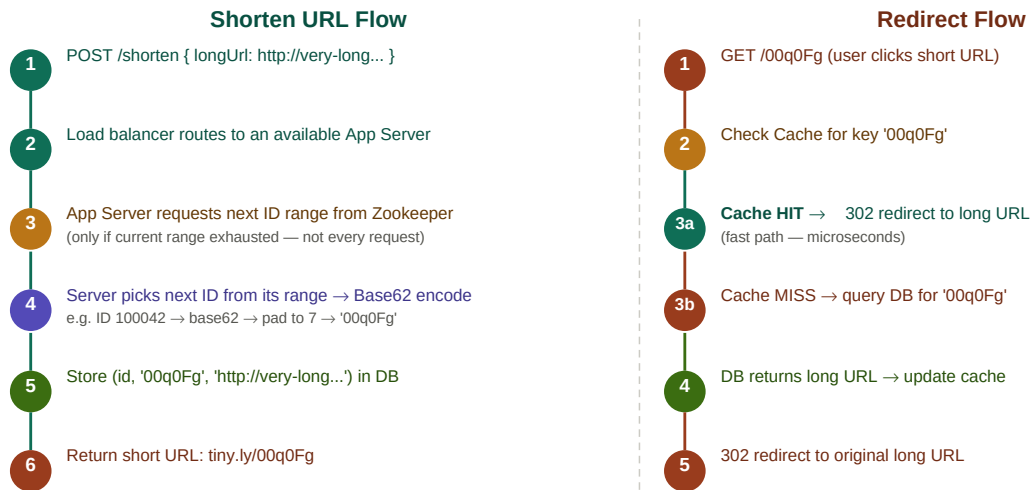


Load Balancer → App Servers → Zookeeper (ID ranges) + Cache (Redis) + DB · Redirect: Cache first, DB on miss

Component	Technology	Role
Load Balancer	NGINX / AWS ALB	Distribute requests across app servers
App Servers	Multiple instances	Handle shorten and redirect logic
Zookeeper	Apache ZooKeeper	Assign unique ID ranges to each server
Cache	Redis / Memcached	short_url → long_url with TTL
Database	MySQL / PostgreSQL	Persistent store: id, short_url, long_url
CDN (optional)	Cloudflare / CloudFront	Cache static assets, reduce latency

5. Request Flow — Shorten & Redirect

Request Flow — Shorten Redirect



301 = permanent redirect (browser caches — reduces server load) · 302 = temporary redirect (hits server every time — better for analytics)

Left: 6-step shorten flow · Right: 5-step redirect flow with cache-hit fast path

6. Database Schema & Design

Database Schema Design Decisions

url_mappings table

Column	Type	Description
id	BIGINT (PK)	Unique numeric ID (Zookeeper / Snowflake)
short_url	VARCHAR(7) UNIQUE	Base62-encoded · indexed for O(1) lookup
long_url	TEXT	Original full URL (up to 2KB)
created_at	TIMESTAMP	Creation time (TTL, analytics)
expires_at	TIMESTAMP (nullable)	Optional URL expiry time

Design Decisions

Read pattern:
Index on short_url for O(1) lookup

Write pattern:
Sequential IDs → good B-tree perf

Scale:
365B rows × ~100 bytes ≈ 36.5 TB
→ shard by id range or hash(short_url)

Cache layer:
short_url → long_url with TTL
80% reads from cache (read-heavy)

Index on short_url (VARCHAR 7) is tiny — entire index fits in memory at scale. Lookup is O(log N) on disk, O(1) from cache.

Simple 5-column table · short_url indexed · read-heavy → cache 80% of lookups · 365B rows ≈ 36.5 TB

7. Quick Revision & Interview Questions

Quick Revision

- URL shortener: long URL → unique short code → redirect. Core challenge = unique ID generation at scale.
- Base62 uses 62 chars (0-9, a-z, A-Z). 7 chars = 3.5 trillion combinations — enough for 100 years.
- MD5/SHA1: output is 32–40 chars. Truncation to 7 chars causes collisions. Not suitable.
- Ticket Server: centralised counter — no collisions but SPOF and bottleneck. Not scalable.
- Snowflake: 64-bit = 41-bit timestamp + 10-bit machine + 12-bit sequence. 4096 IDs/ms/machine.
- Zookeeper: divides 3.5T ID space into ranges per server. Zero collisions, fully distributed.

- DB schema: id (PK), short_url (UNIQUE, indexed), long_url, created_at, expires_at.
- System is read-heavy (~100:1). Cache (Redis) stores short→long with TTL. 80% served from cache.
- 301 = permanent redirect (browser caches, reduces load). 302 = temporary (every request hits server, good for analytics).
- Padding: base62 IDs shorter than 7 chars are left-padded with "0" for uniform length.
- Capacity: $10\text{M/day} \times 365 \times 100 = 365 \text{ billion URLs}$. Each row ~100 bytes → ~36.5 TB total storage.
- Sharding: shard DB by id range or hash(short_url) when single DB hits limits.

Interview Questions

- 1. Design a URL shortener like TinyURL. Walk through the full system.
- 2. Why is 7 characters the minimum length? Show the calculation.
- 3. Why can't we use MD5 or SHA1 directly for URL shortening?
- 4. What is Base62 encoding? How do you convert an integer to a Base62 string?
- 5. What is a Ticket Server? What are its limitations?
- 6. Explain Twitter's Snowflake ID algorithm. What are its components?
- 7. How does Zookeeper help with unique ID generation in a distributed system?
- 8. What is the difference between 301 and 302 redirects? When do you use each?
- 9. How would you design the database for this system? What indexes are needed?
- 10. How would you handle 10 billion redirect requests per day?
- 11. How does the cache layer work in a URL shortener? What is the eviction strategy?
- 12. How would you support custom short URLs (e.g., tiny.ly/my-brand)?
- 13. What happens if two servers request the same ID range from Zookeeper simultaneously?

Key Terms

URL Shortener	Service that maps a long URL to a unique short code for redirection
Base62	62-character encoding (0-9, a-z, A-Z) for compact ID representation
Collision	Two different long URLs mapping to the same short code — must be avoided
Ticket Server	Centralised counter that issues sequential IDs — SPOF and bottleneck
Snowflake ID	64-bit distributed ID: 41-bit timestamp + 10-bit machine + 12-bit sequence
Zookeeper	Distributed coordination service that assigns ID ranges to servers
ID Range	Block of IDs assigned to one server — server generates from this block without coordination
Padding	Left-padding short Base62 strings with "0" to enforce 7-char uniform length
301 Redirect	Permanent redirect — browser caches it, subsequent visits skip the server
302 Redirect	Temporary redirect — every visit hits server, useful for analytics
Read:Write ratio	~100:1 for URL shorteners — most traffic is redirects, not shortenings
SPOF	Single Point of Failure — one component whose failure brings down the system